

CIS 5800

Machine Perception

Or Geometric Computer Vision

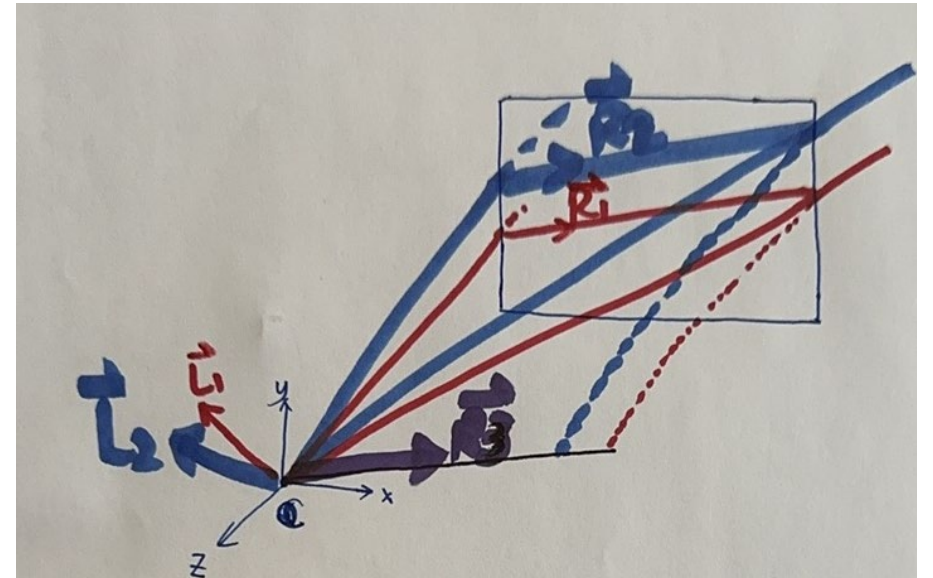
Instructor: Lingjie Liu

Lec 4: Feb 3, 2026

Recap: Extending to Any Parallel Lines

- Under projective geometry,
 - All parallel lines intersect at a point at infinity
 - One point at infinity $\Leftrightarrow$ one parallel line direction

line $l = (a, b, c)^T$ intersects at $(b, -a, 0)^T$

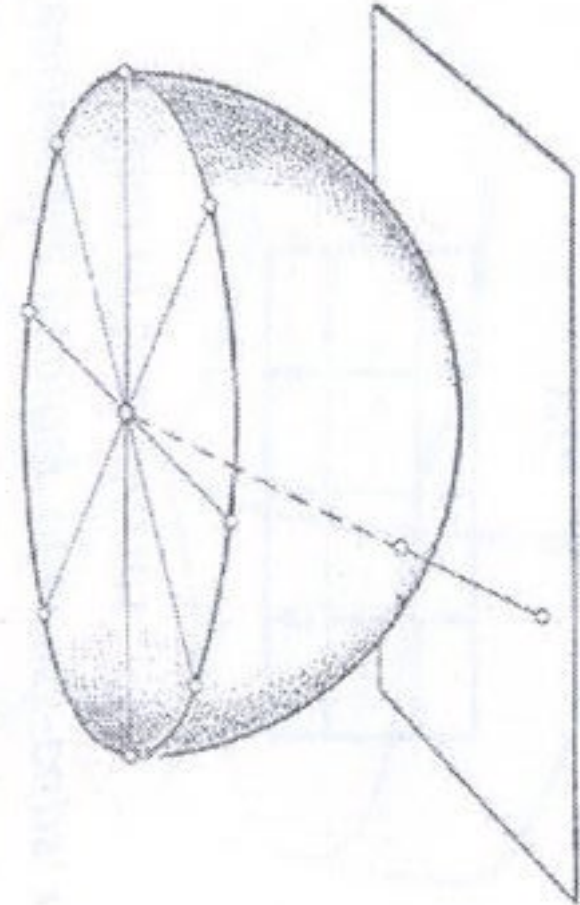


Recap: Point at infinity / “ideal” points

$(x_1, x_2, 0)$

Looking-at direction

“Ideal” points



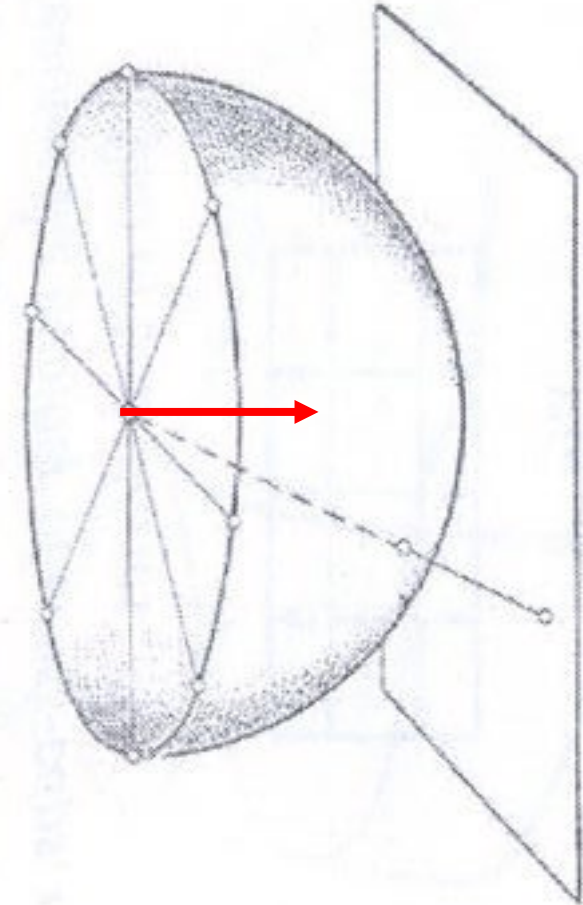
Recap: “Line at infinity”

- A line passing through all ideal points i.e. point

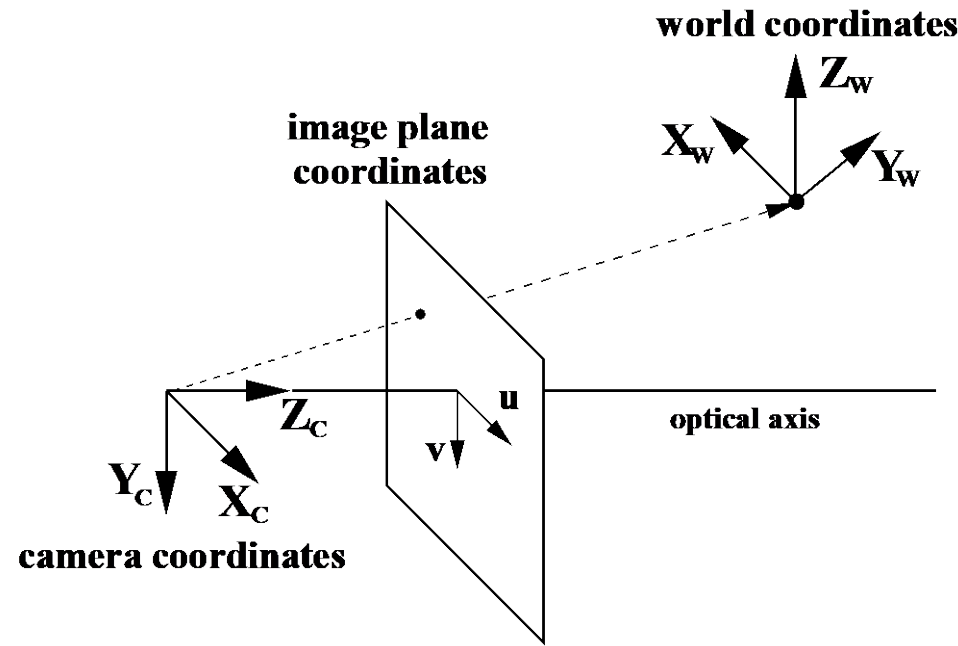
$$l_{\infty} = (0, 0, 1)^T$$

- Because :

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ 0 \end{bmatrix} = 0$$



Recap: Putting the pieces together: Projection matrix P



camera 3D coords to pixels Convert world to camera coordinates

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & & u_0 \\ & f & v_0 \\ & & 1 \end{bmatrix} \begin{bmatrix} R_{3 \times 3} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = K[R|\mathbf{t}]\mathbf{X}_w$$

Recap: Perspective Projection in homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} = P_{3 \times 4} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\underbrace{P_{3 \times 4}}_{\substack{\text{"Camera} \\ \text{projection} \\ \text{matrix"}}} = \underbrace{K_{3 \times 3}}_{\text{"Intrinsics"}} \underbrace{[R_{3 \times 3} | \mathbf{t}_{3 \times 1}]}_{\text{"Extrinsics"}}$$

Q: How many degrees of freedom does P have given the type of intrinsics we have assumed?

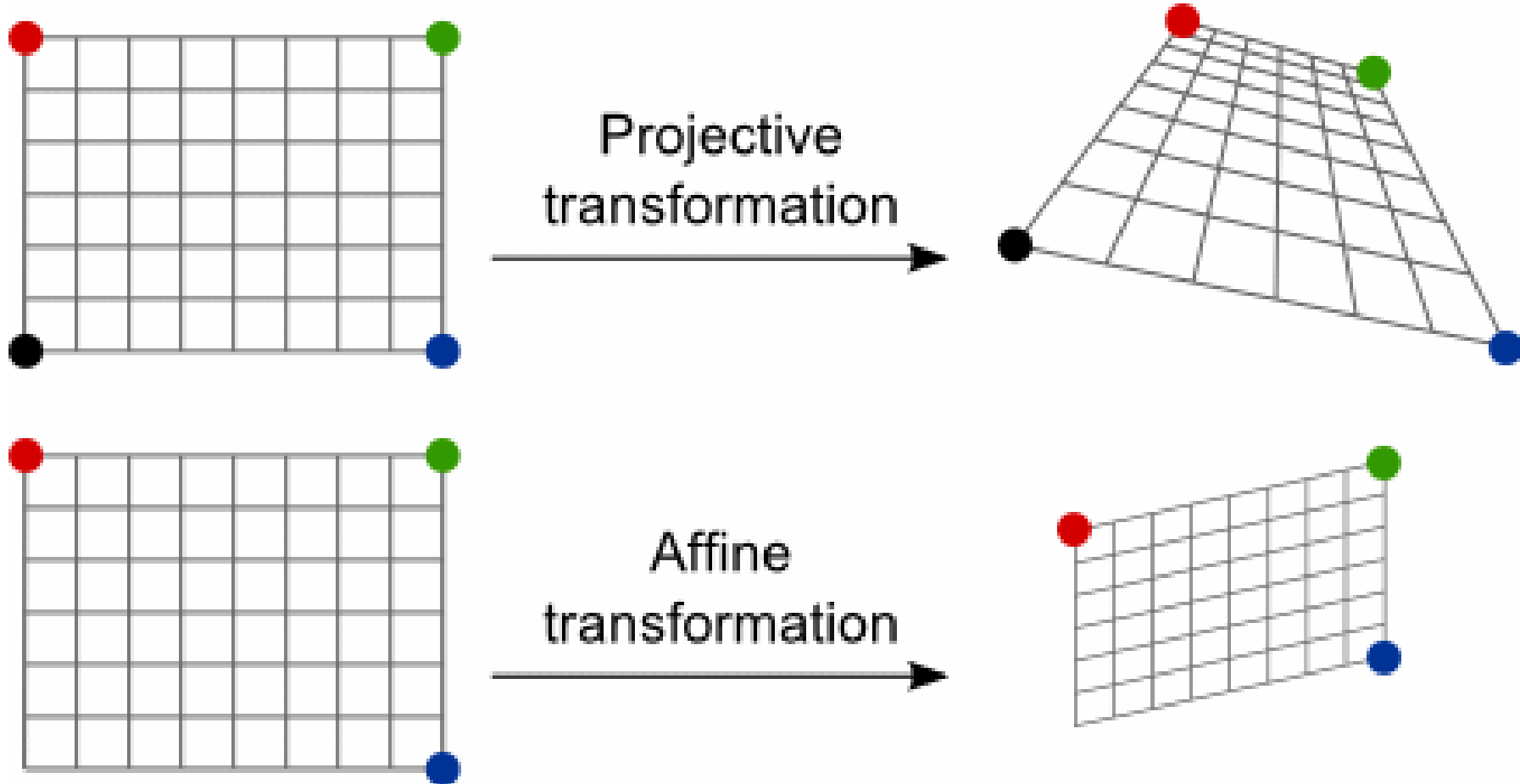
Projective Transformations

aka Collineations
aka Homographies
aka Projectivity

Example of Projective Transformation

Common notations: H

(Note that some books use A ; however, we will avoid using A in this course, as A is commonly associated with Affine Transformations.)



Example of Projective Transformation

- A 2D point before H is represented as (X, Y) , after Projective transformation is (u, v) :

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \sim H \begin{pmatrix} X \\ Y \\ 1 \end{pmatrix} \quad \text{or}$$

$$u = \frac{H_{11}X + H_{12}Y + H_{13}}{H_{31}X + H_{32}Y + H_{33}}$$
$$v = \frac{H_{21}X + H_{22}Y + H_{23}}{H_{31}X + H_{32}Y + H_{33}}$$

Projective Transformation = Homography = Collineation=Projectivity

Definition

A **projective transformation** is any invertible matrix transformation $\mathbb{P}^2 \rightarrow \mathbb{P}^2$.

Projective Transformation = Homography = Collineation=Projectivity

Definition

A **projective transformation** is any invertible matrix transformation $\mathbb{P}^2 \rightarrow \mathbb{P}^2$.

A projective transformation H maps p to $p' \sim Hp$

Invertibility means that $\det(H) \neq 0$ and that there exists $\lambda \neq 0$ such that $\lambda p' = Hp$

Observe that we will write either $p' \sim Hp$ or $\lambda p' = Hp$

How many unknowns are in a projective transformation H ?

$$(\mathbb{P}^2 \rightarrow \mathbb{P}^2)$$

A projective transformation μH is the same as H since they map to projectively equivalent points:

$$\mu \lambda p' = \mu H p$$

How many unknowns are in a projective transformation H ?

$$(\mathbb{P}^2 \rightarrow \mathbb{P}^2)$$

A projective transformation μH is the same as H since they map to projectively equivalent points:

$$\mu \lambda p' = \mu H p$$

We will be able to determine a projective transformation only up to a scale factor. Hence the 3x3 invertible matrix H will have only EIGHT independent unknowns.

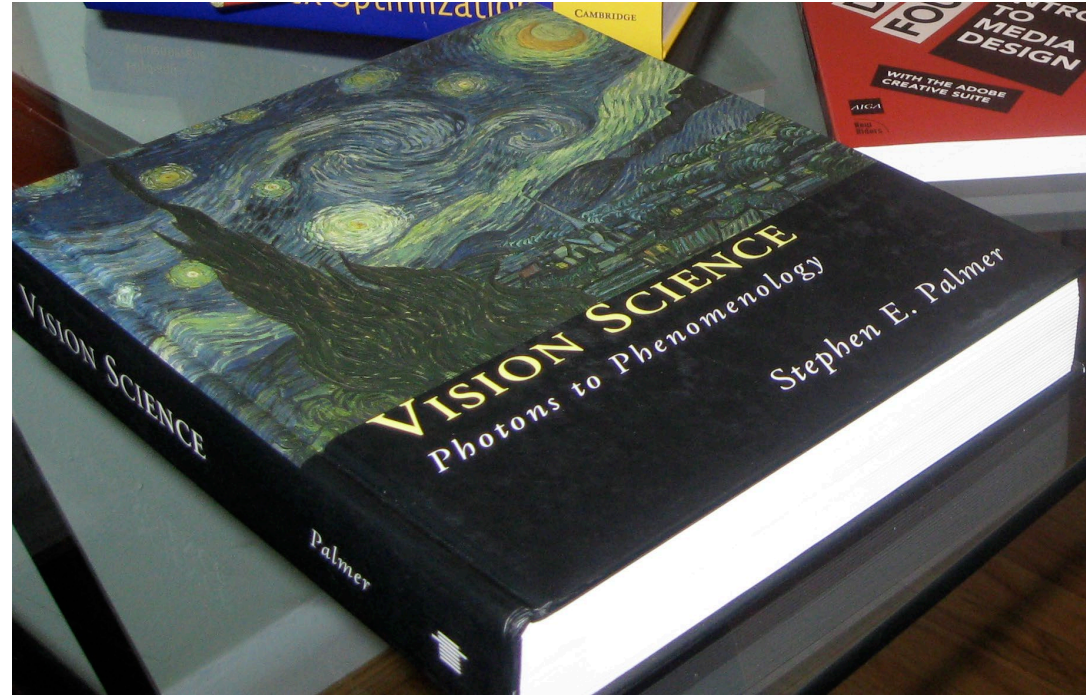
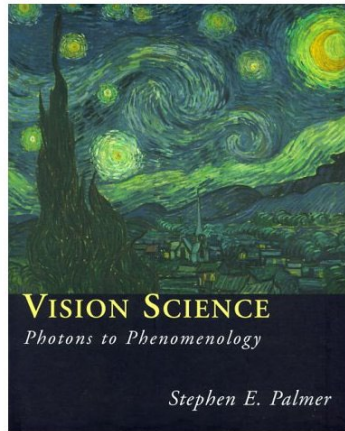
Q: How many unknowns in a projective transformation in $\mathbb{P}^3$?

Perspective Projection v.s. Projective Transformation

	Perspective Projection	Projective Transformation
Definition	A mapping from 3D space to a 2D plane (e.g., camera image)	A general mapping between projective space (e.g., P^2 to P^2)
Mathematical Formula	$p' = K[R T]P$	$p' = H \cdot p$
Input Space	R^3 (can also be P^3)	P^n (typically P^2 in this class)
Output Space	R^2 (can also be P^2)	P^n (typically P^2 in this class)
Applications	Image formulation, 3D rendering	Image registration, planar transformation, texture mapping

When Perspective Projection -> Projective Transformation?

A perspective camera projection of a plane (i.e., a camera image) is a projective transformation in $\mathbb{P}^2$

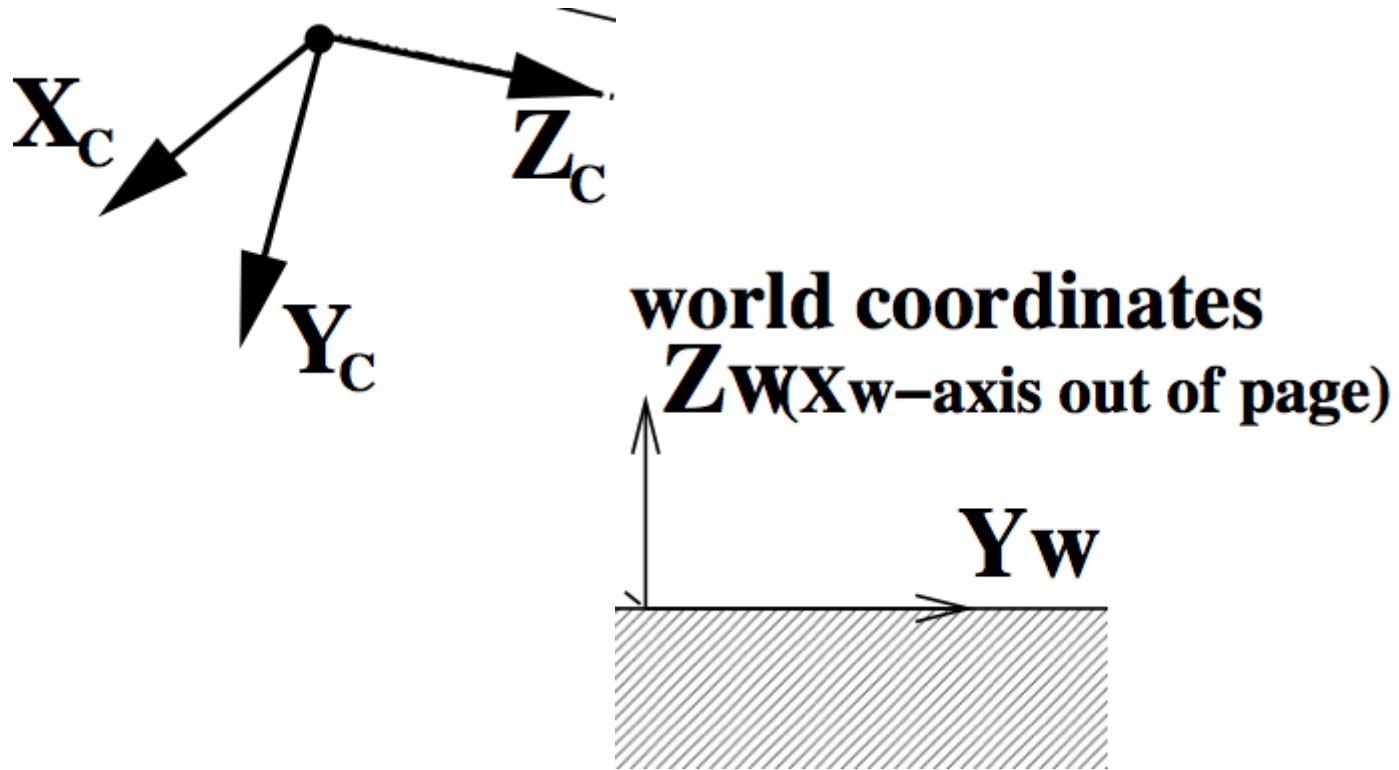


When Perspective Projection $\rightarrow$ Projective Transformation?

- Can we show that the perspective camera projection from $\mathbb{P}^3 \rightarrow \mathbb{P}^2$ of a plane in the world is in fact a homography in $\mathbb{P}^2$ (i.e., projective transformation from $\mathbb{P}^2 \rightarrow \mathbb{P}^2$) when the world plane coordinates are expressed in $\mathbb{P}^2$?
- Remember:

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \sim K_{3 \times 3} [R_{3 \times 3} | \mathbf{t}_{3 \times 1}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Assume world plane $Z_w = 0$



When Perspective Projection -> Projective Transformation?

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane $Z = 0$.

Pose From Homography

Recall the projection from world to camera

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim K \begin{pmatrix} r_1 & r_2 & r_3 & T \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix}$$

and assume that all points in the world lie in the ground plane $Z = 0$.

Then the transformation reads

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} \sim \underbrace{K \begin{pmatrix} r_1 & r_2 & T \end{pmatrix}}_{\text{Homography}} \begin{pmatrix} X \\ Y \\ W \end{pmatrix}$$

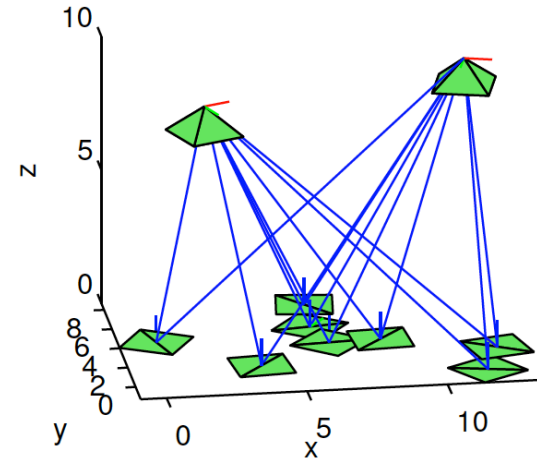
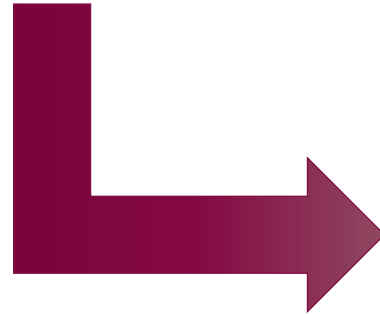
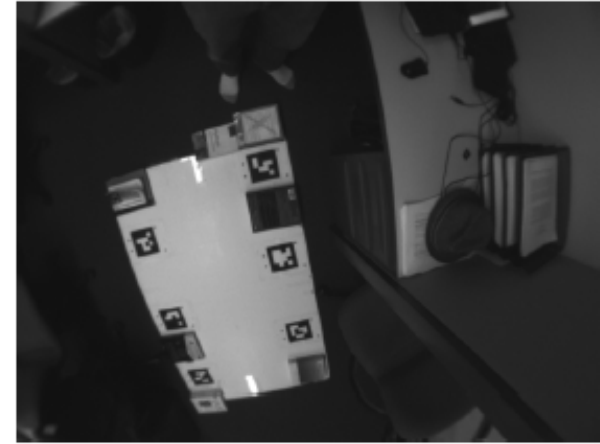
The planar homography
 $H: \mathbb{P}^2 \rightarrow \mathbb{P}^2$

Computing the homography can tell us how the camera (and therefore, e.g. a robot attached to the camera) is oriented w.r.t. to a world plane! (assuming known K)

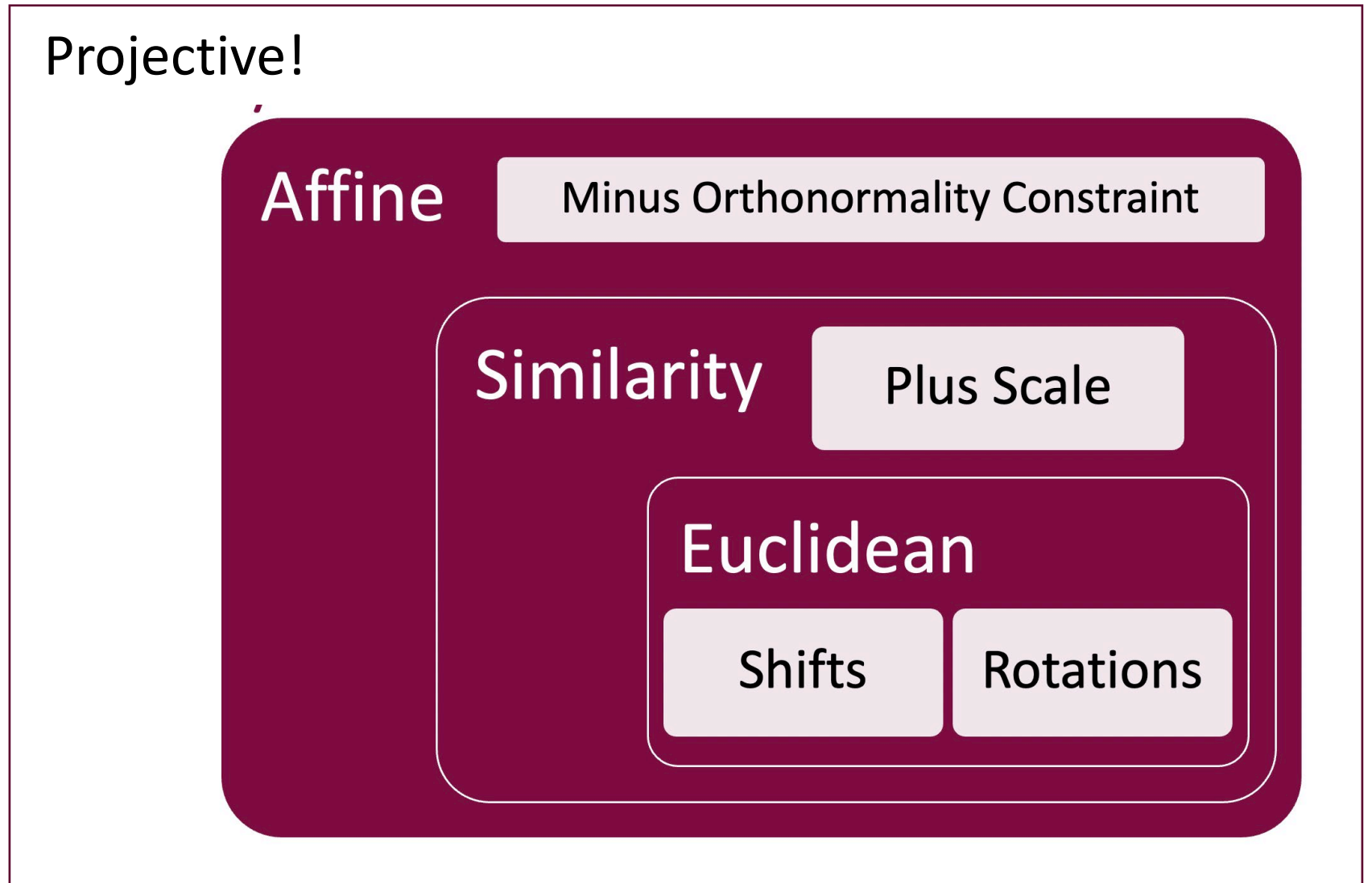
Q: Where do you get r_3 from though?

A: $r_3 = r_1 \times r_2$

Localization w.r.t. known planes using homographies

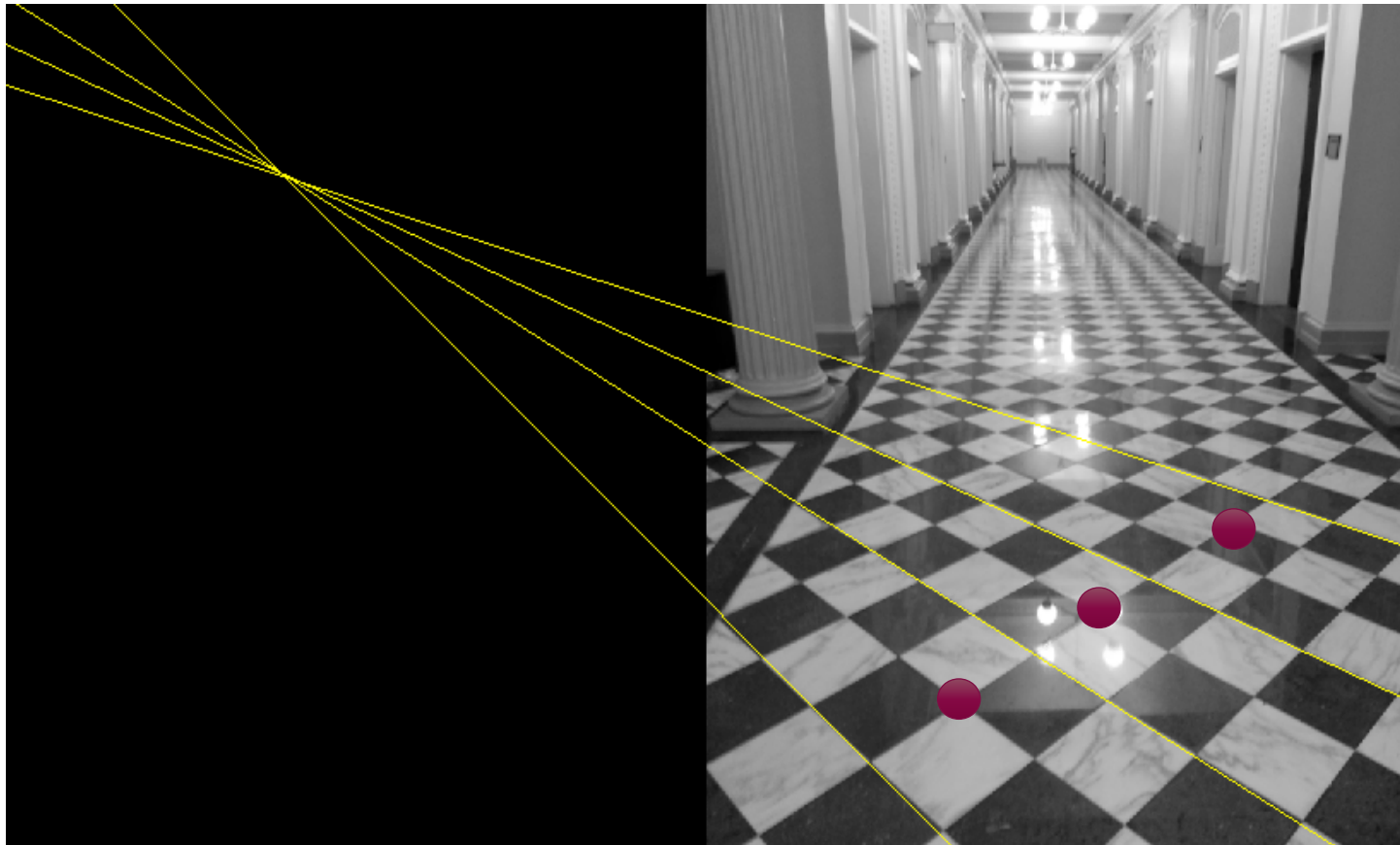


Place in the Hierarchy of Transformations

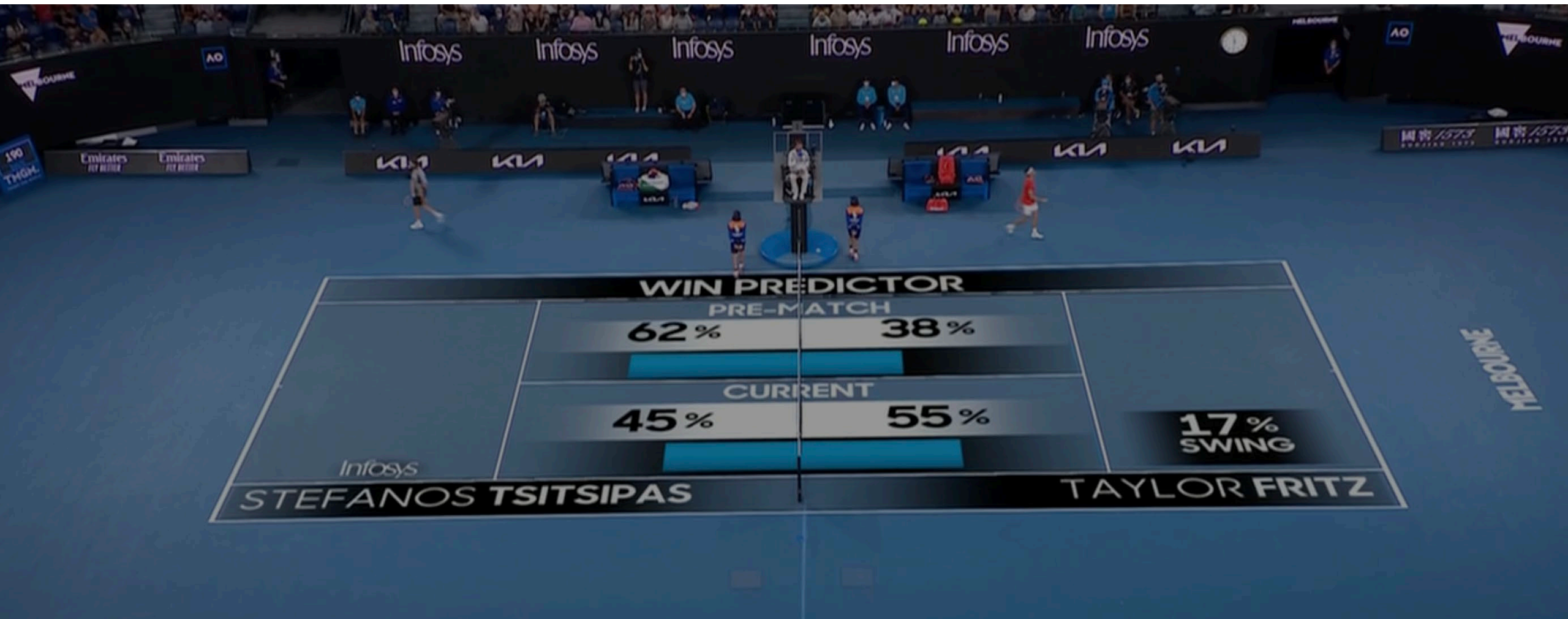


A projective transformation preserves incidence:

- Three collinear points are mapped to three collinear points.
- and three concurrent lines are mapped to three concurrent lines.



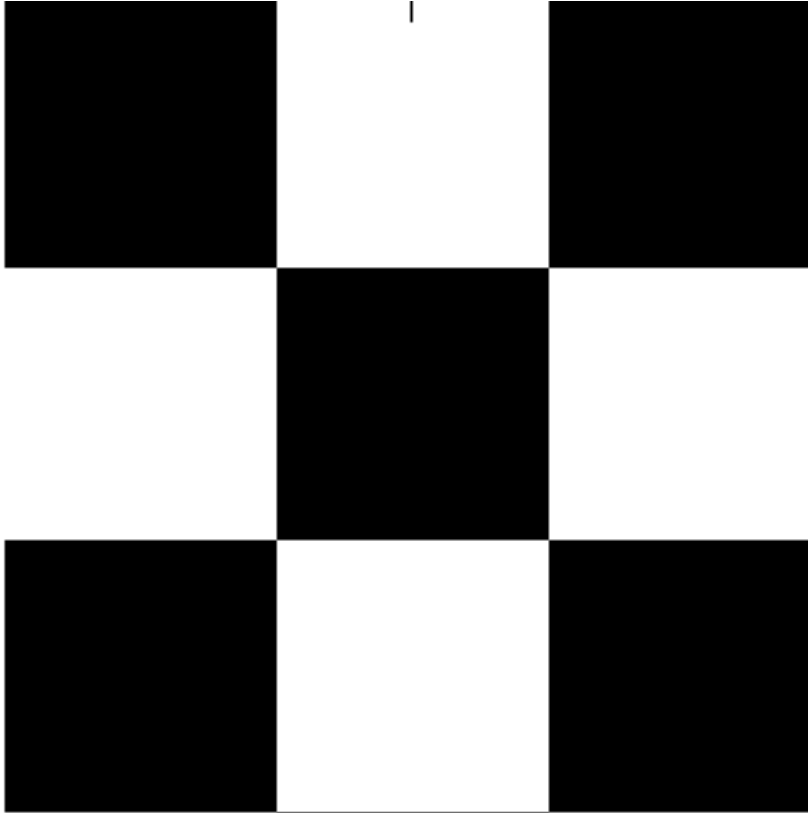
Application Example: Virtual Billboards



Computing Homographies From 4 Point Correspondences

“4-point collineation”

How can we compute the projective transformation between a known pattern and its projection?



Floor tiles measured in [m]

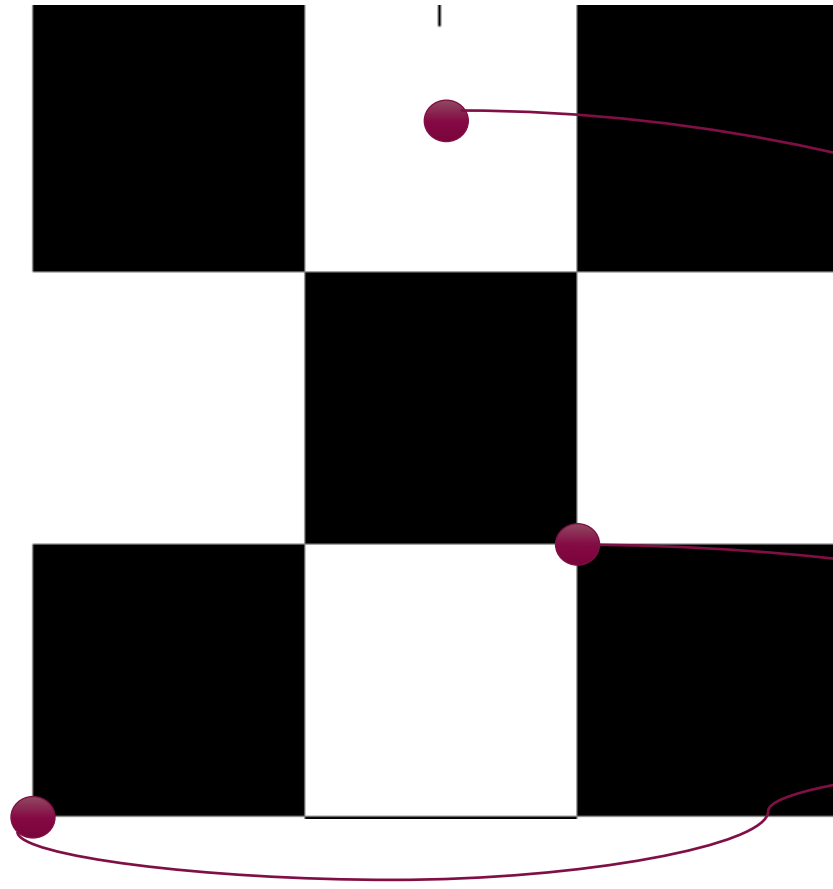
and



Points in pixel coordinates

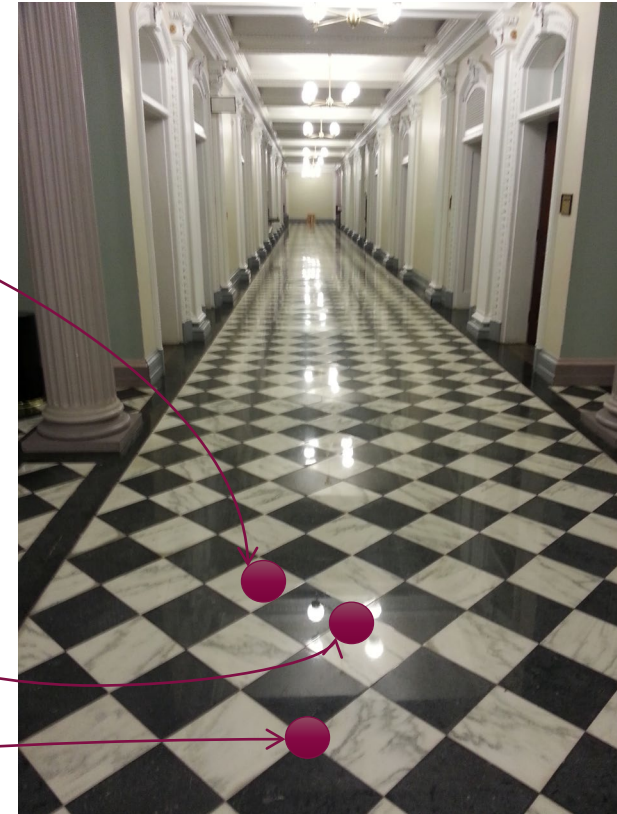
The result of such a transformation would map any point in one plane to the corresponding point in the other

“correspondences”



Floor tiles measured in [m]

and



Points in pixel coordinates

Recap: **How many unknowns are in a projective transformation H ?**

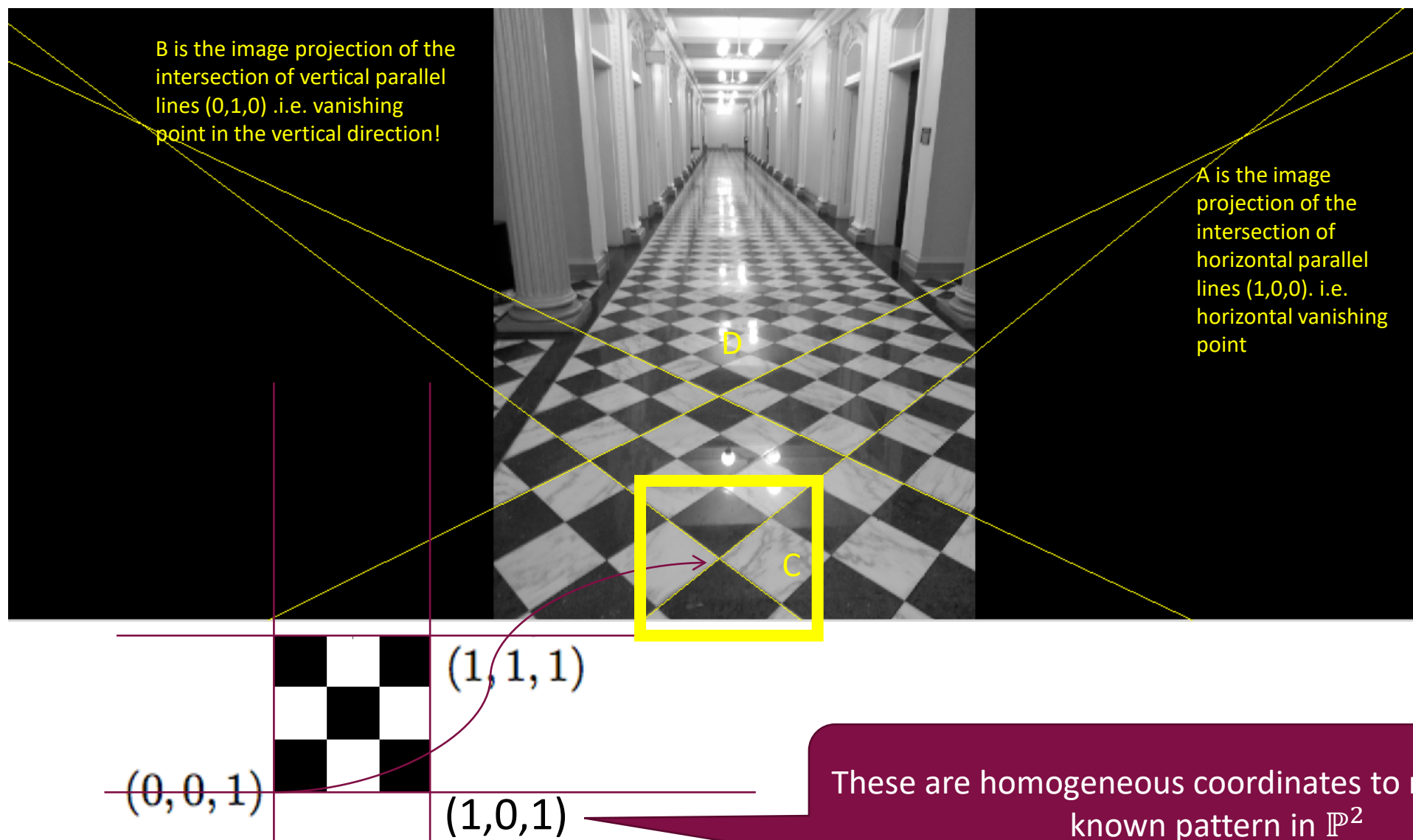
$$(\mathbb{P}^2 \rightarrow \mathbb{P}^2)$$

A projective transformation μH is the same as H since they map to projectively equivalent points:

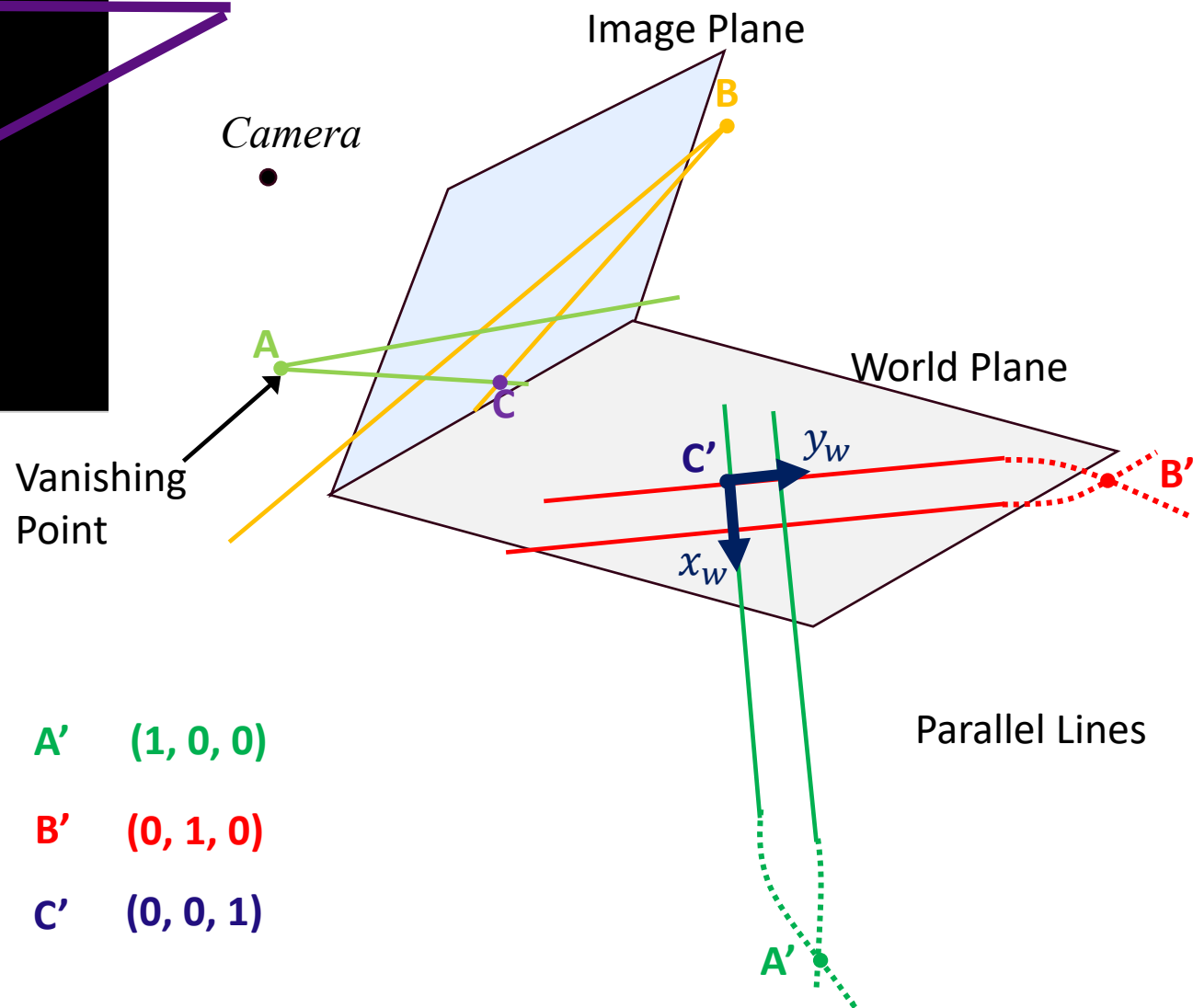
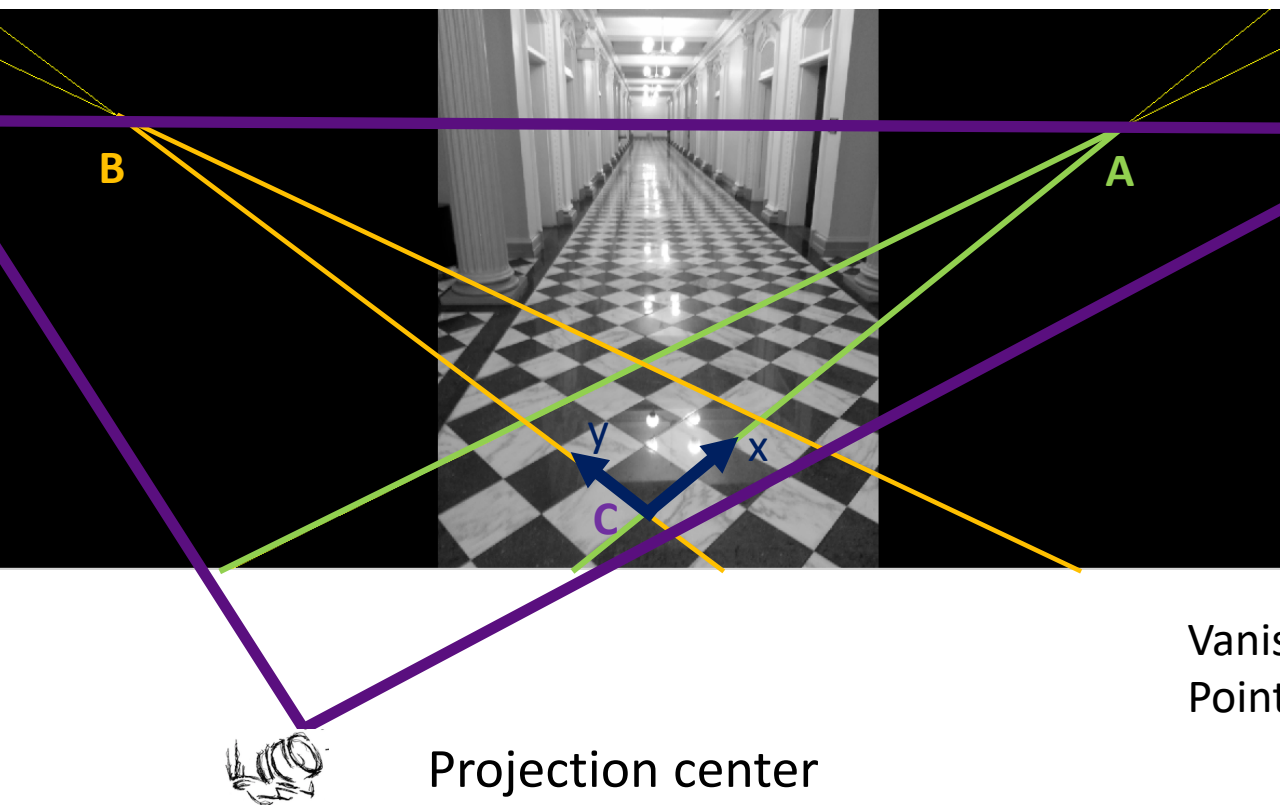
$$\mu \lambda p' = \mu H p$$

We will be able to determine a projective transformation only up to a scale factor. Hence the 3x3 invertible matrix H will have only EIGHT independent unknowns.

How can we compute the projective transformation between a known pattern and its projection?



Cont.



Correspondences:
($P^2 \rightarrow P^2$)

A	a	$\longleftrightarrow$	A'	(1, 0, 0)
B	b	$\longleftrightarrow$	B'	(0, 1, 0)
C	c	$\longleftrightarrow$	C'	(0, 0, 1)

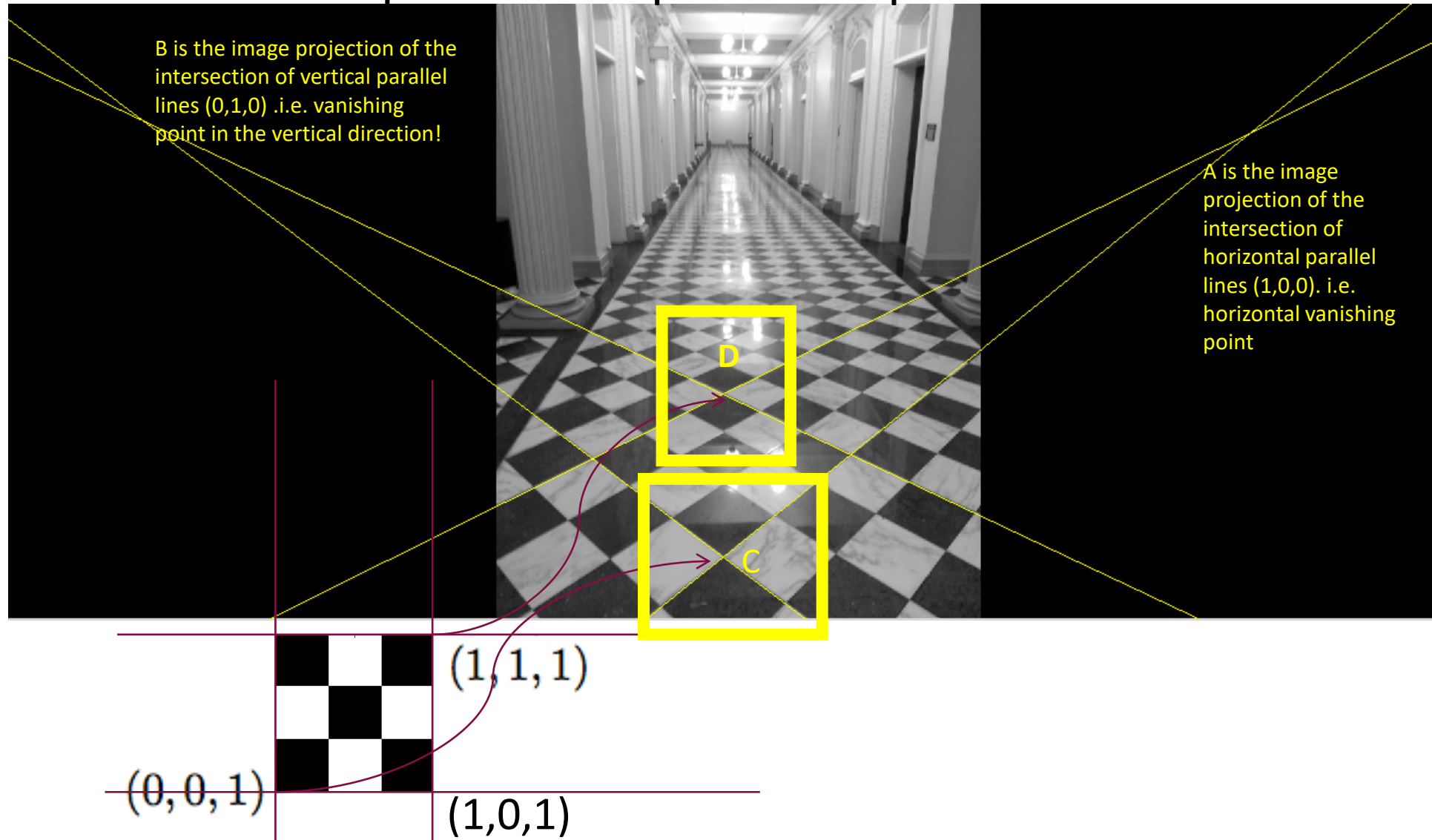
Assume that a mapping H maps the three points $(1, 0, 0)$, $(0, 1, 0)$, and $(0, 0, 1)$ to the non-collinear points A, B, C

with coordinate vectors a, b and $c \in \mathbb{P}^2$. Then the following is a possible projective transformation:

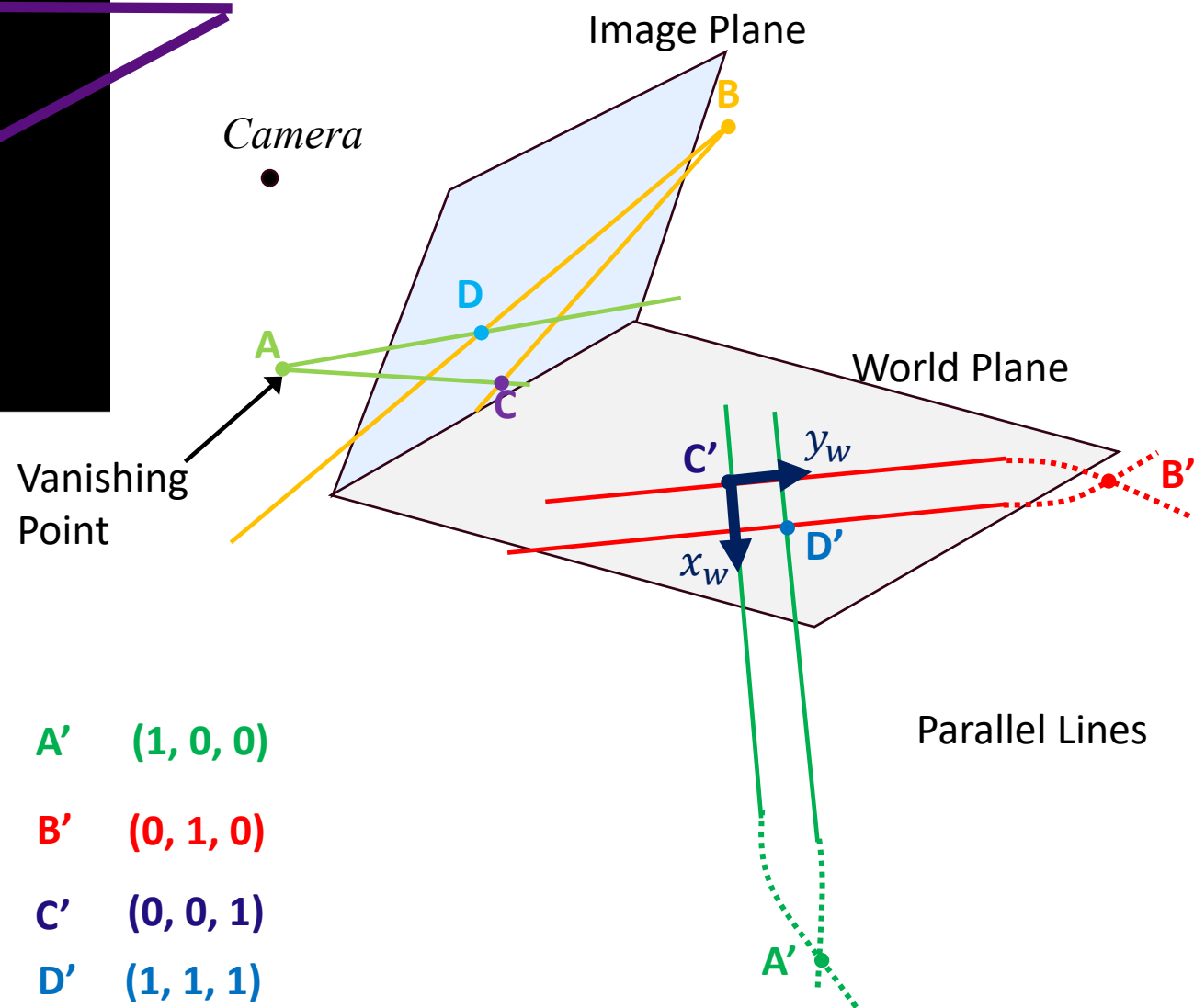
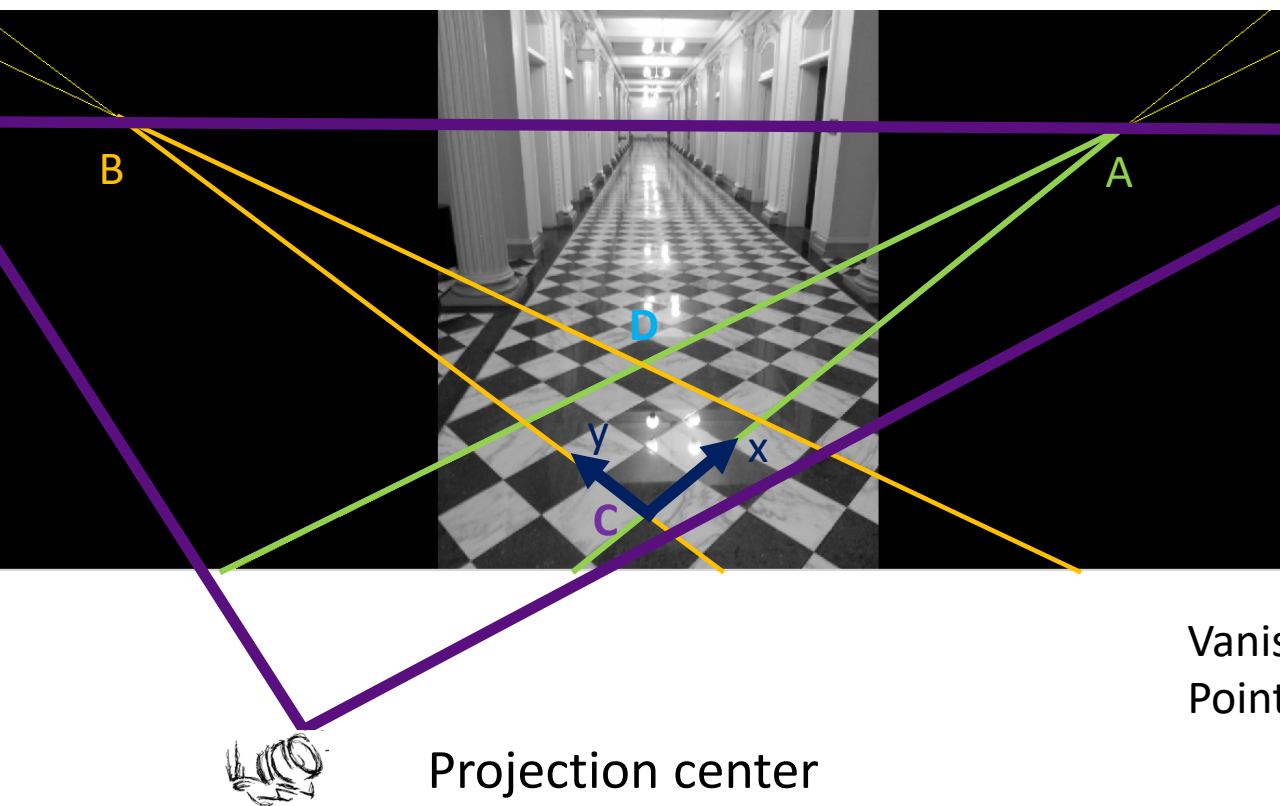
$$\begin{array}{c} \textcolor{green}{A} \quad \textcolor{blue}{B} \quad \textcolor{red}{C} \\ \left(\begin{array}{|c|} \hline \textcolor{green}{a} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{blue}{b} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{red}{c} \\ \hline \end{array} \right) \sim \textcolor{red}{H}_{3 \times 3} \left(\begin{array}{|c|} \hline \textcolor{green}{1} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{red}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{blue}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{green}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{red}{1} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{blue}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{green}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{red}{0} \\ \hline \end{array} \begin{array}{|c|} \hline \textcolor{blue}{1} \\ \hline \end{array} \right)$$

Solution: Introduce a 4th point correspondence D

Note: makes sense, because after all, H has 8 degrees of freedom, and each 2D point correspondence pins down 2DOF.



Cont.



Correspondences:
($P^2 \rightarrow P^2$)

A	a	$\longleftrightarrow$	A'	(1, 0, 0)
B	b	$\longleftrightarrow$	B'	(0, 1, 0)
C	c	$\longleftrightarrow$	C'	(0, 0, 1)
D	d	$\longleftrightarrow$	D'	(1, 1, 1)

Let us assume that the same H maps $(1, 1, 1)$ to the point d . Then, the following should hold:

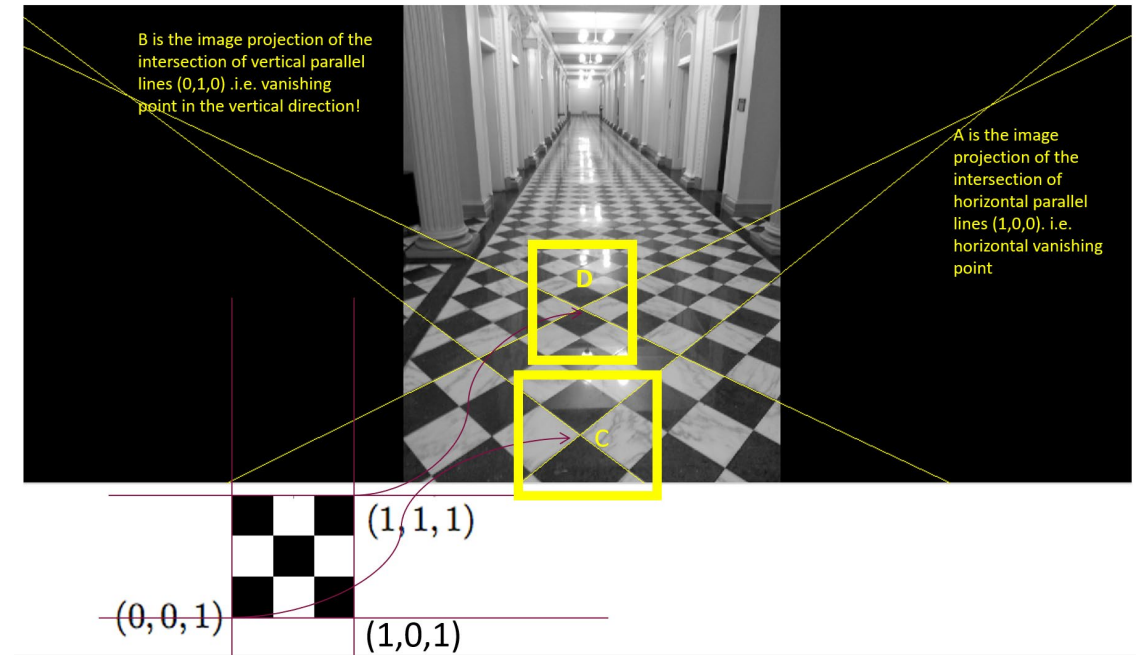
$$d \sim \underset{H_{3 \times 3}}{\begin{pmatrix} \alpha a & \beta b & \gamma c \end{pmatrix}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix},$$

hence

$$d \sim \alpha a + \beta b + \gamma c. \quad \text{or} \quad \lambda d = \alpha a + \beta b + \gamma c$$

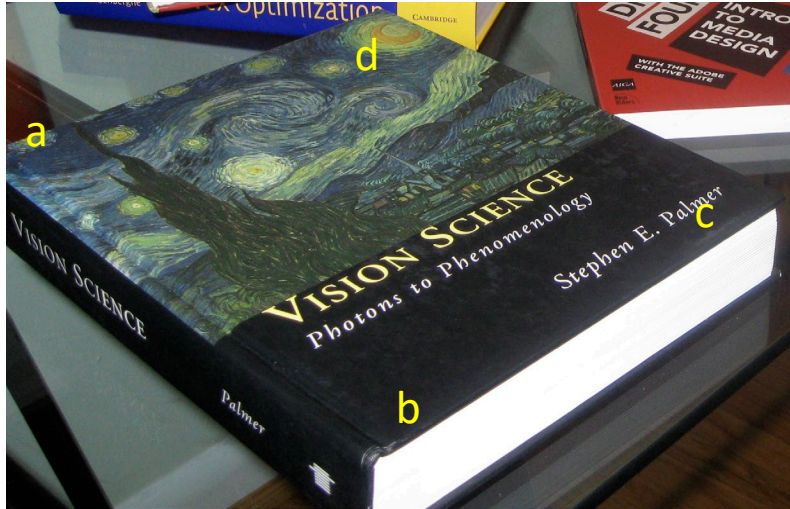
Because a, b, c are not collinear, there exist unique $\alpha/\lambda, \beta/\lambda, \gamma/\lambda$ for writing this linear combination.

Four points, no three of them collinear, suffice to unambiguously recover a homography



Choosing the points to be the horizontal and vertical vanishing points $(1,0,0)$, $(0,1,0)$ plus origin $(0,0,1)$ and the diagonal $(1,1,1)$ is particularly “nice” especially if you have a square to start from, but really, any four non-collinear points will do.
(coming up next)

What happens when the original set of points is not a square?



Find projective transformation mapping $(a, b, c, d) \rightarrow (a', b', c', d')$:

To determine this mapping we go through the four canonical points.

We find the mapping from $(1, 0, 0), etc$ to (a, b, c, d) and we call it T :

$$a \sim T(1, 0, 0)^T, etc$$

We find the mapping from $(1, 0, 0), etc$ to (a', b', c', d') and we call it T' :

$$a' \sim T'(1, 0, 0)^T, etc$$

Then, back-substituting $(1, 0, 0)^T \sim T^{-1}a, etc$ we obtain that

$$a' = T'T^{-1}a, etc$$

To determine this mapping we go through the four canonical points.

We find the mapping from $(1, 0, 0), etc$ to (a, b, c, d) and we call it T :

$$a \sim T(1, 0, 0)^T, etc$$

We find the mapping from $(1, 0, 0), etc$ to (a', b', c', d') and we call it T' :

$$a' \sim T'(1, 0, 0)^T, etc$$

Then, back-substituting $(1, 0, 0)^T \sim T^{-1}a, etc$ we obtain that

$$a' = T'T^{-1}a, etc$$

To determine this mapping we go through the four canonical points.

We find the mapping from $(1, 0, 0)$, *etc* to (a, b, c, d) and we call it T :

$$a \sim T(1, 0, 0)^T, \text{ etc}$$

We find the mapping from $(1, 0, 0)$, *etc* to (a', b', c', d') and we call it T' :

$$a' \sim T'(1, 0, 0)^T, \text{ etc}$$

Then, back-substituting $(1, 0, 0)^T \sim T^{-1}a$, *etc* we obtain that

$$a' = T'T^{-1}a, \text{ etc}$$

This amounts to computing homographies from a made-up square, then inverting one of them and composing!

Homography -> Virtual Billboards

- For virtual billboards, we just treat the desired billboard pattern as the world pattern.
- Recall that the homography is a mapping from world to image pixel coordinates. So, just the homography can directly find the image pixel coordinate corresponding to every image in the world pattern.

(Q: is this actually what we need for virtual billboards? A: not quite.)



Microsoft Office Lens App



Office Lens

Microsoft Corporation Productivity

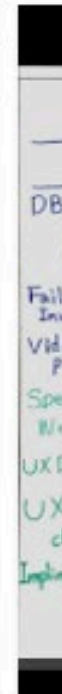
Everyone

This app is compatible with all of your devices.

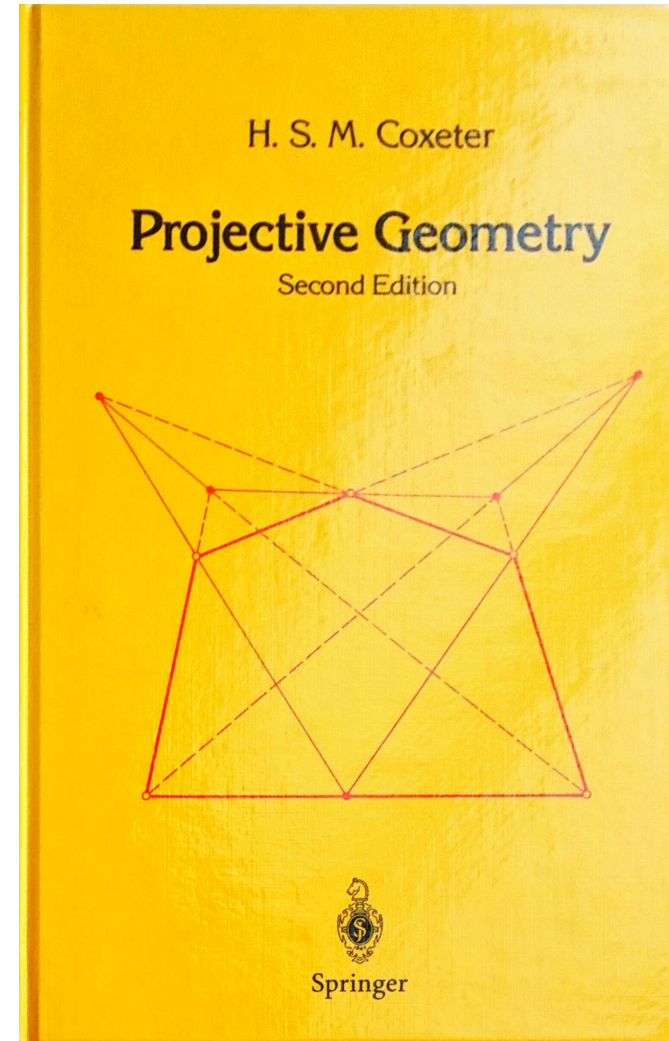
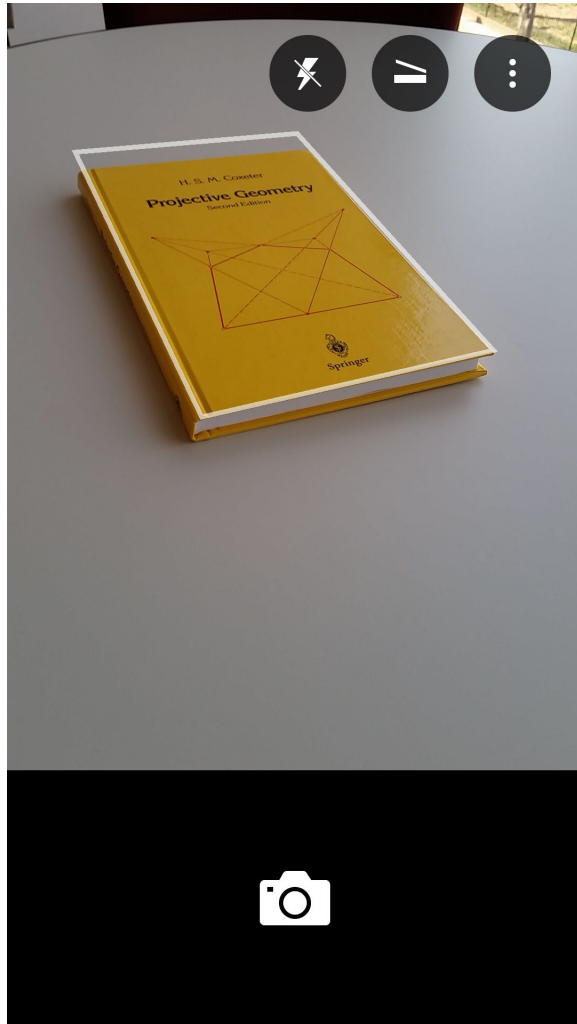
Top Developer

★★★★★ 91,419

Installed



Office Lens



Computing homographies with more correspondences

$$\mathbf{x}' \sim H\mathbf{x}$$

$$\lambda \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\lambda x' = h_{11}x + h_{12}y + h_{13}$$

$$\lambda y' = h_{21}x + h_{22}y + h_{23}$$

$$\lambda = h_{31}x + h_{32}y + h_{33}$$

Converting to a linear system of equations

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$



$$\begin{aligned} -h_{11}x - h_{12}y - h_{13} + h_{31}xx' + h_{32}yx' + h_{33}x' &= 0 \\ -h_{21}x - h_{22}y - h_{23} + h_{31}xy' + h_{32}yy' + h_{33}y' &= 0 \end{aligned}$$



Two linear equations for
each point correspondence!

$$\begin{pmatrix} a_x \\ a_y \end{pmatrix} h = 0$$

$$a_x = \begin{pmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \end{pmatrix}$$

$$a_y = \begin{pmatrix} 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{pmatrix}$$

$$h = \begin{pmatrix} h_{11} & h_{12} & h_{13} & h_{21} & h_{22} & h_{23} & h_{31} & h_{32} & h_{33} \end{pmatrix}^T$$

Computing homographies with more correspondences

Our matrix H has 8 degrees of freedom, and so, as each point gives 2 sets of equations, we will need 4 points to solve for h uniquely. So, given four points (such as the corners provided for this assignment), we can generate vectors a_x and a_y for each, and concatenate them together:

$$A = \begin{pmatrix} a_{x,1} \\ a_{y,1} \\ \vdots \\ a_{x,n} \\ a_{y,n} \end{pmatrix} \quad 2n \times 9$$

$$Ah = 0$$

When $n > 4$, due to noise in measurements, there may not be an $h \neq 0$ that produces exactly $Ah = 0$.

Instead, we find the vector s.t. $\|Ah\|_2$ is smallest.

This involves computing the SVD (singular value decomposition) of A , and setting h to be the “smallest” right singular vector, with smallest singular value.

(more info on SVD etc. soon)

